

Project Design Phase

Solution Architecture

Date	28 September 2026
Team ID	SWTID-2026-2670
Project Name	BlogNest
Maximum Marks	4 Marks

Solution Architecture

The BlogNest solution architecture describes how the client/request layer communicates with the Express API, how authentication and authorization protect routes, how controllers process requests, and how Mongoose models store and retrieve data from MongoDB.

Figure 1: BlogNest Architecture and Data Flow

Client/Frontend	HTTP Request	Express API / Routes	Controllers + Middleware	MongooseDB / Mongoose Models
User Login • Posts • Comments • Categories • AI	GET / POST / PUT / DELETE JSON request + response	authRoutes blogRoutes commentRoutes categoryRoutes userRoutes analyticsRoutes aiRoutes	JWT Role Controllers: Auth / Blog / Comment / Category Category / User / Analytics / AI	protect() User authorize() Post Comment Analytics

Major Components

Layer	Project Components	Responsibility
Server	server.js, app.js	Loads environment configuration, connects to database, and starts the Express server.
Routes	auth, blog, category, comment, user, analytics, AI routes	Defines API endpoints and applies protection/role authorization where required.
Middleware	authMiddleware.js, errorHandler.js	Validates JWT bearer tokens, loads the user, checks roles, and formats errors.
Controllers	authController, blogController, categoryController, commentController, userController, analyticsController, aiController	Contains application/business logic for each feature.
Data Layer	Mongoose models: User, Post, Comment, Category, Analytics	Defines schemas and persists application data in MongoDB.
Utilities / Security	generateToken.js; bcryptjs; helmet; cors; rate-limit; validator	Supports JWT creation, password hashing, HTTP security, CORS, rate limiting, and validation dependencies.

Functional Data Flow

1. User/client sends an HTTP request to a BlogNest API endpoint.
2. Express routes receive the request and, for protected endpoints, the protect middleware checks the Bearer JWT.
3. The authorization middleware checks whether the authenticated user's role is permitted for the requested operation.
4. The relevant controller validates/processes the request and interacts with the Mongoose model.
5. MongoDB stores or retrieves the required data through Mongoose.
6. The controller returns a JSON response containing success/message/data or an error response.

Implemented API Feature Groups

- Authentication: register, login, current-user retrieval.
- Blog: list posts, get a post, create/update/delete posts, like/unlike.
- Categories: list, create, update, and delete categories with admin/editor protection.

- **Comments:** create comments, list comments for authorized roles, update status, and delete.
- **Users:** list users for admin/ editor and update roles for admin.
- **Analytics:** protected analytics retrieval.
- **AI:** protected blog generation, summarization, and FAQ endpoints.

Development / Delivery Requirements

- Configure environment variables such as database connection, JWT secret, server port, and AI service credentials where required.
- Connect the Express server to MongoDB before starting the API.
- Use authentication tokens for protected endpoints and appropriate user roles for administrative operations.
- Test each route with valid and invalid requests before integrating with a frontend.
- Keep the modular route/controller/model structure when adding future features.

The supplied code shows a modular Node.js/Express backend using MongoDB through Mongoose and JWT-based access control.

Reference structure: Solution Architecture template supplied by the user. `filecite` `turn1file0` `L2-L19`